

Машинное обучение

Лекция 3

Линейная регрессия. Градиентный спуск.

Михаил Гущин

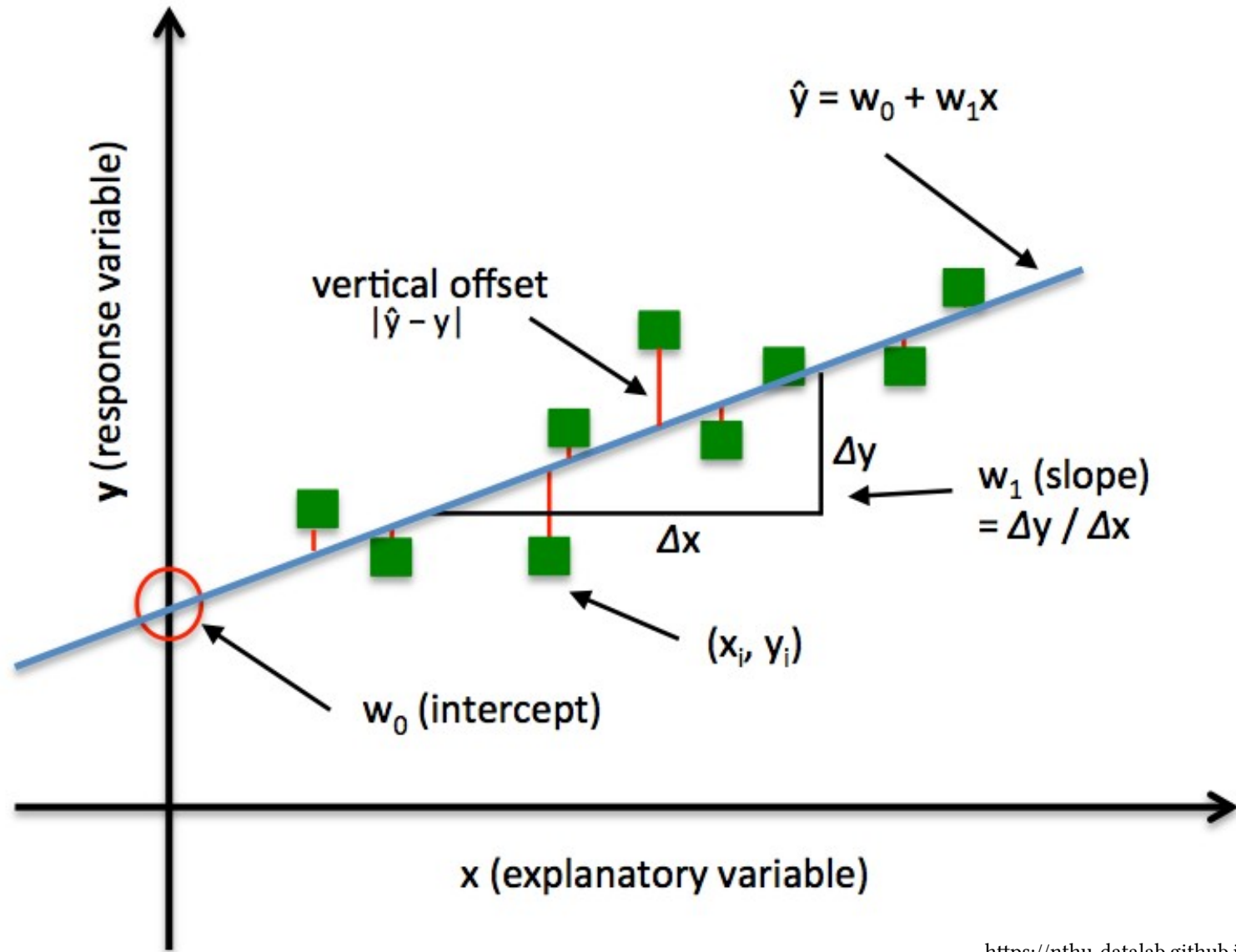
mhushchyn@hse.ru

НИУ ВШЭ, 2026

Линейная регрессия. Аналитическое решение.



Линейная регрессия



<https://nthu-datalab.github.io>

Матричная форма

- ▶ Модель линейной регрессии:

$$\hat{y} = Xw$$

- $X = \begin{pmatrix} \mathbf{1} & x_{11} & \cdots & x_{1d} \\ \vdots & & \ddots & \vdots \\ \mathbf{1} & x_{n1} & \cdots & x_{nd} \end{pmatrix}$ - матрица признаков объектов $(n, d + 1)$;
- $w = (w_0, w_1, \dots, w_d)^T$ - вектор $(d + 1)$ весов модели;
- $\hat{y} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n)^T$ - вектор прогнозов модели для (n) объектов;

- ▶ Вектор квадратов ошибок прогнозов модели: $(\hat{y} - y)^2$

Аналитическое решение

►
$$L = (\hat{y} - y)^T (\hat{y} - y) = (Xw - y)^T (Xw - y)$$

Чтобы найти минимум L , надо:

$$\frac{\partial L}{\partial w} = 0$$

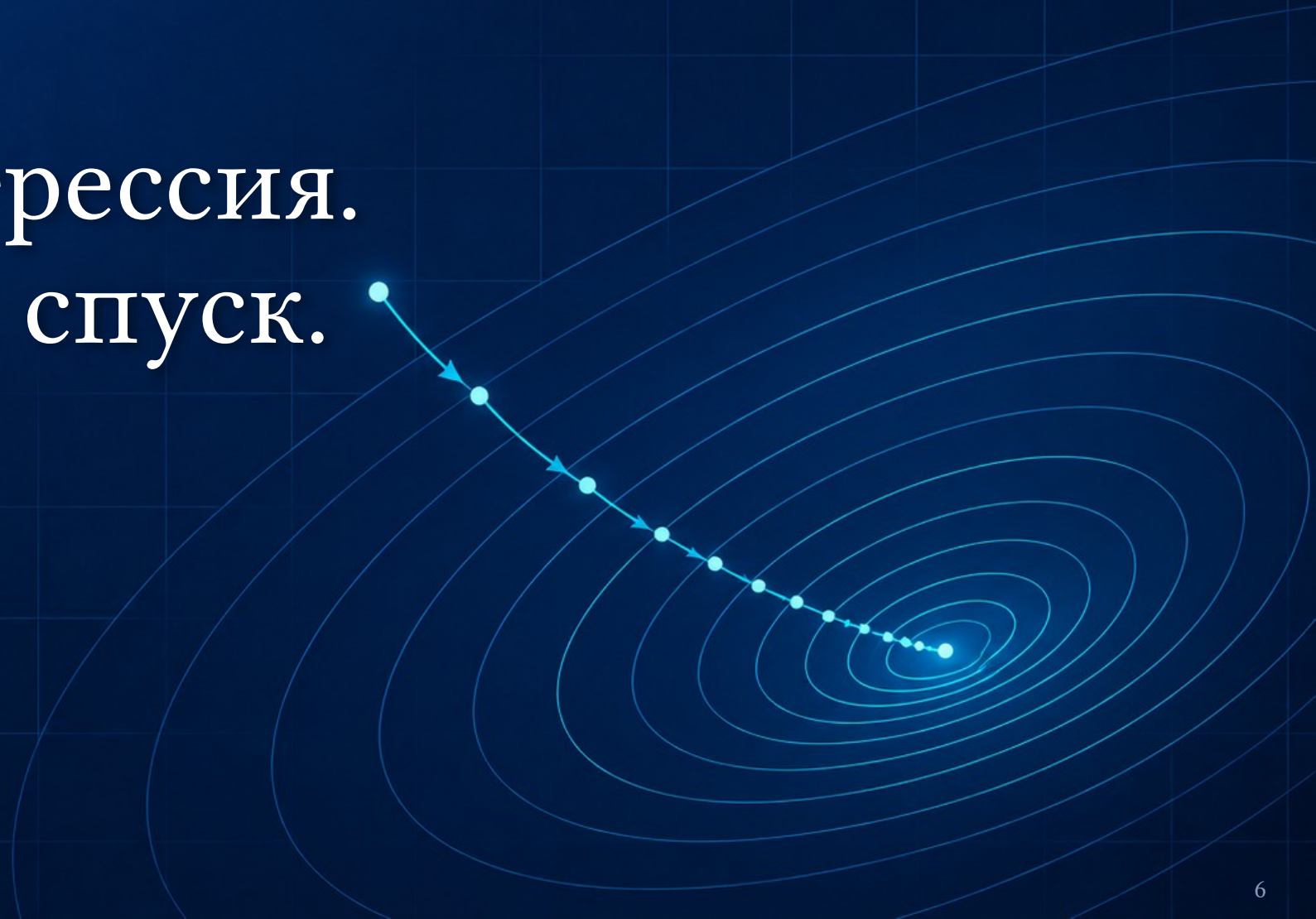
Тогда

$$\frac{\partial L}{\partial w} = 2X^T (Xw - y) = 2X^T Xw - 2X^T y = 0$$

Получаем оптимальные веса w линейной регрессии:

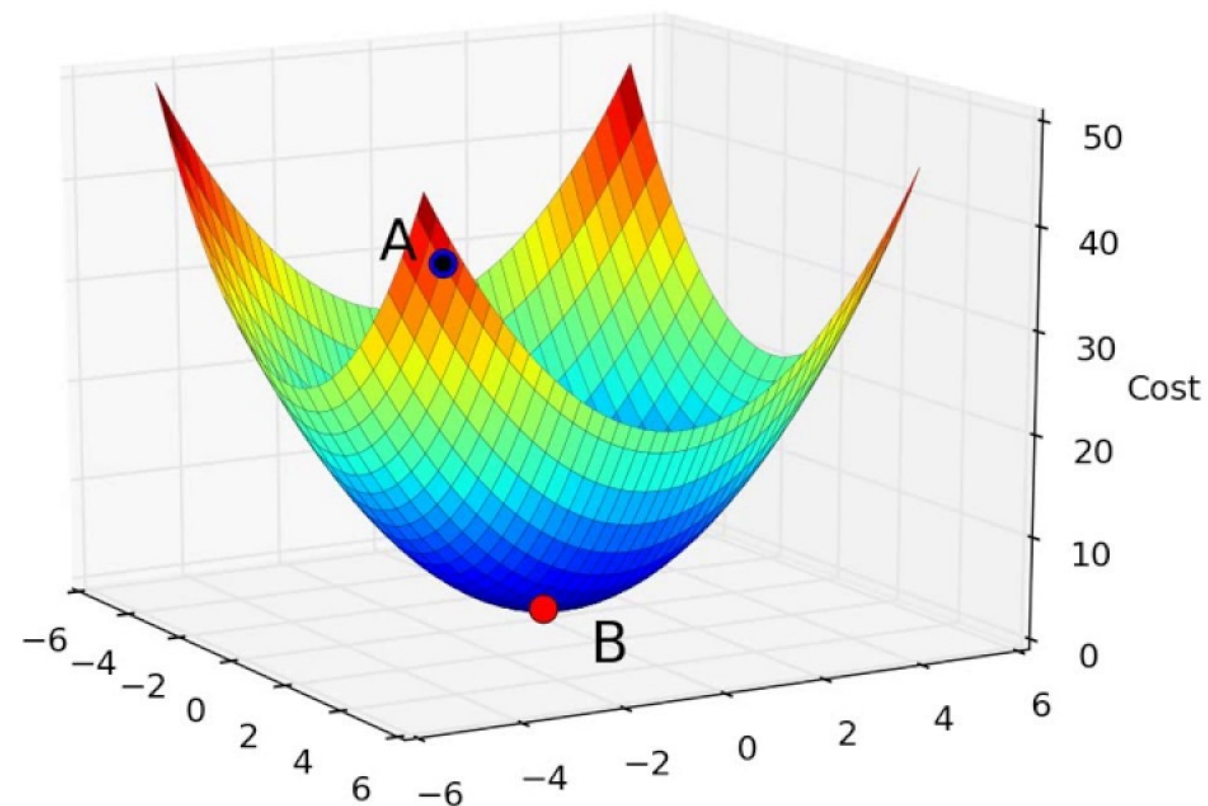
$$w = (X^T X)^{-1} X^T y$$

Линейная регрессия. Градиентный спуск.



Задача

- ▶
- ▶ Есть функция $L(w)$
- ▶ Хотим найти ее минимум:
$$L \rightarrow \min_w$$
- ▶ Мы умеем считать ее производную $\frac{\partial L}{\partial w}$
- ▶ Но не умеем (или не хотим) решать уравнение $\frac{\partial L}{\partial w} = 0$



Градиент функции

- ▶ Градиент функции (∇L) – вектор первых частных производных функции:

$$\nabla L(w_0, w_1, \dots, w_d) = \left(\frac{\partial L}{\partial w_0}, \frac{\partial L}{\partial w_1}, \dots, \frac{\partial L}{\partial w_d} \right)$$

- ▶ В векторной форме мы будем писать:

$$\nabla L(w) = \frac{\partial L(w)}{\partial w}$$

Градиентный спуск

- ▶ Есть функция $L(w)$, минимум которой хотим найти
- ▶ Пусть $w^{(0)}$ - начальный вектор параметров. Например, $w^{(0)} = 0$
- ▶ Тогда **градиентный спуск** состоит в повторении:

$$w^{(k+1)} = w^{(k)} - \eta \nabla L(w^{(k)})$$

- η – длина шага градиентного спуска (**learning rate**) (мы сами его задаем)
- k – номер итерации
- $\nabla L(w^{(k)})$ – градиент функции потерь на итерации k

Пример

- ▶ Модель линейной регрессии:

$$\hat{y} = Xw$$

- ▶ Функция потерь MSE:

$$L = \frac{1}{n}(\hat{y} - y)^T(\hat{y} - y) = \frac{1}{n}(Xw - y)^T(Xw - y)$$

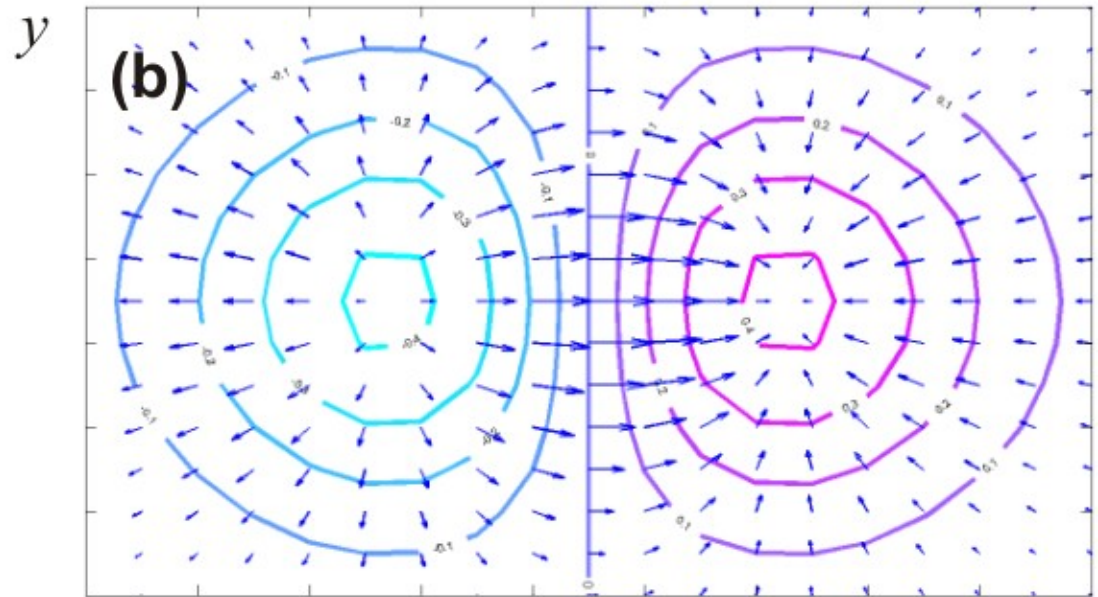
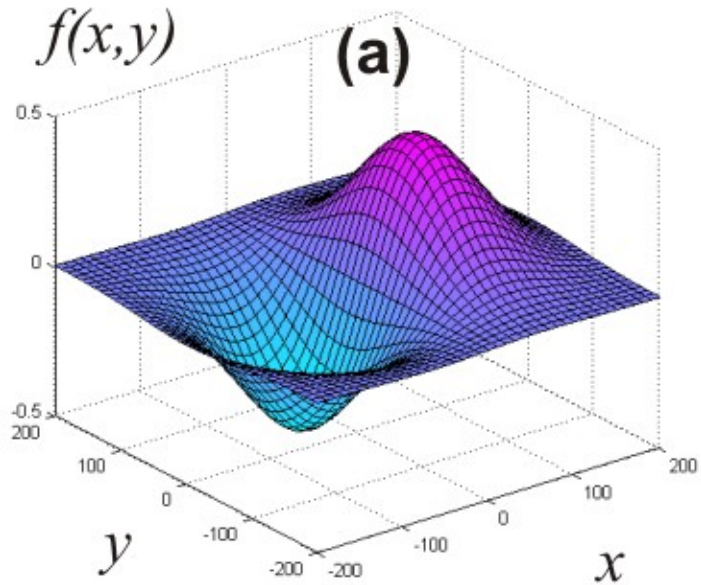
- ▶ Градиент:

$$\nabla L(w) = \frac{2}{n}X^T(Xw - y)$$

- ▶ Градиентный спуск:

$$w^{(k+1)} = w^{(k)} - \eta \nabla L(w^{(k)}) = w^{(k)} - \eta \frac{2}{n}X^T(Xw^{(k)} - y)$$

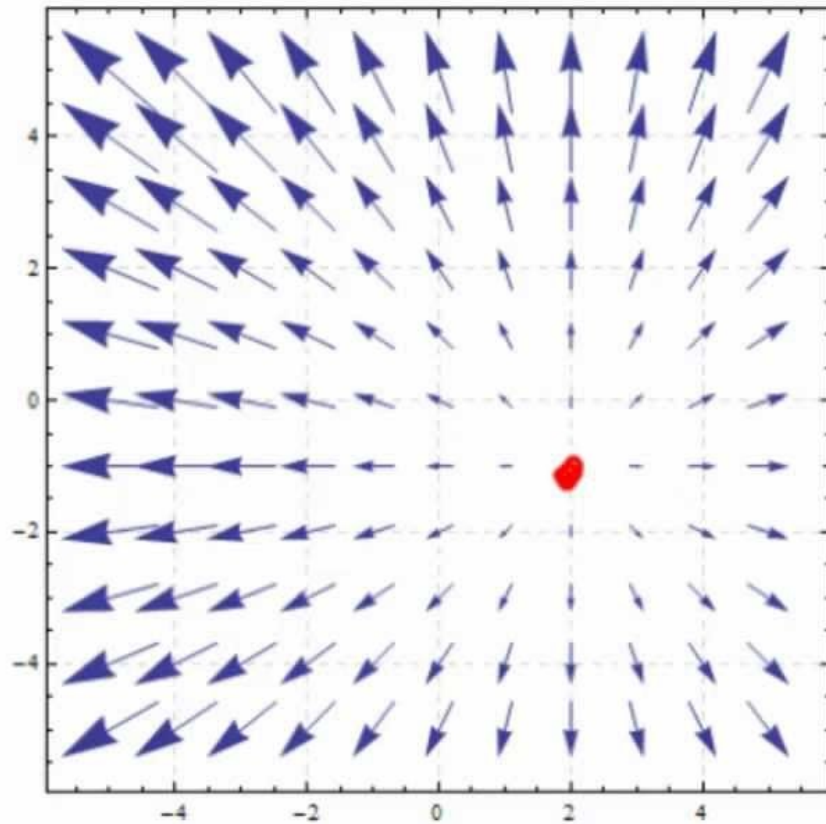
Свойства градиента



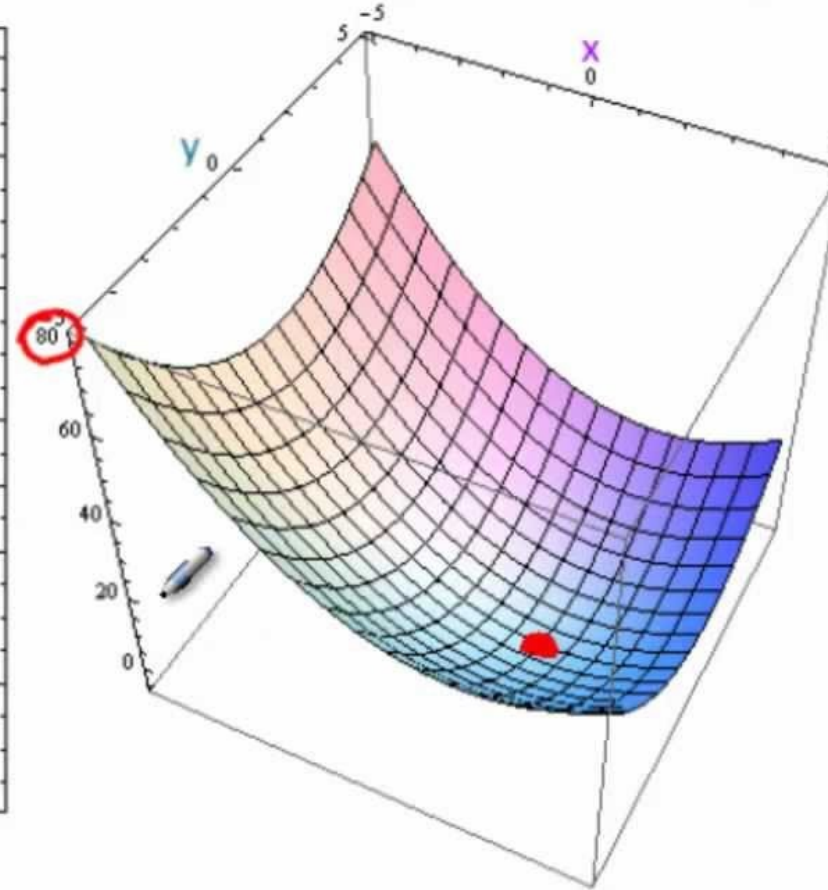
Карта градиентов и линии уровня
функции

Свойства градиента

The gradient field $\langle 2x-4, 2y+2 \rangle$

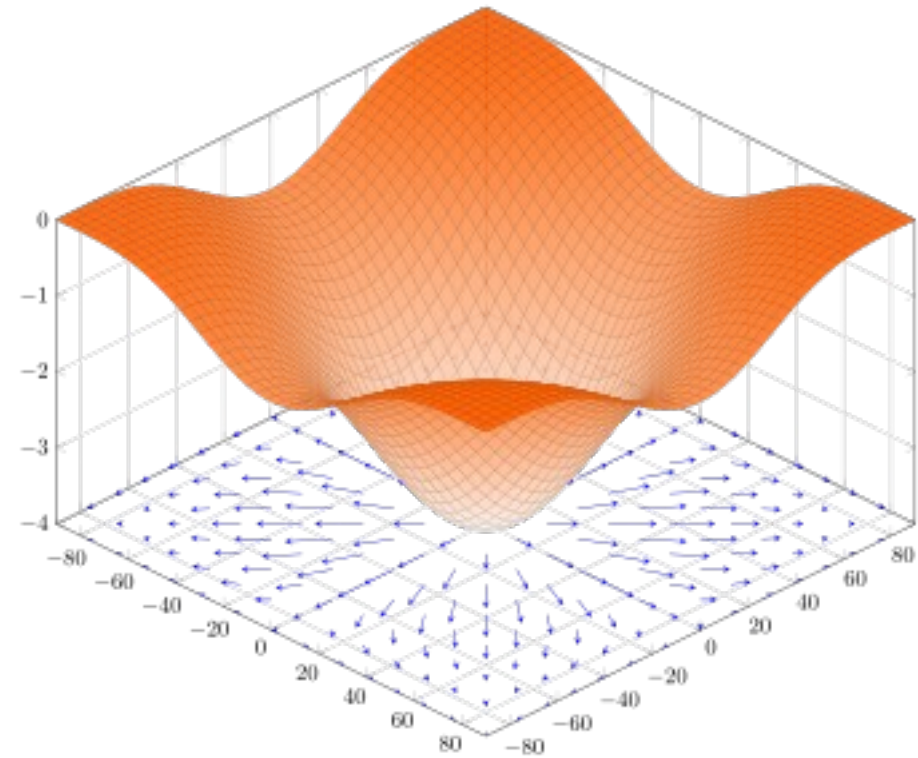


of the function $f = x^2 - 4x + y^2 + 2y$.



Свойства градиента

- ▶ Градиент функции в некоторой точке ортогонален линии уровня, проходящей через эту точку
- ▶ Градиент функции указывает направление наискорейшего возрастания функции в данной точке
- ▶ Направление **анти**градиента указывает направление наискорейшего убывания функции в данной точке

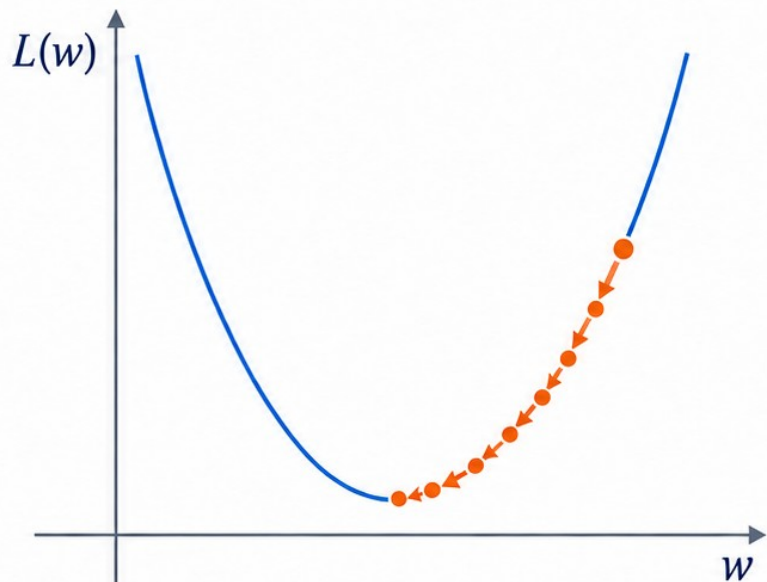


Градиентный спуск

- ▶ Как выбрать длину градиентного спуска?
- ▶ Сколько итераций делать?

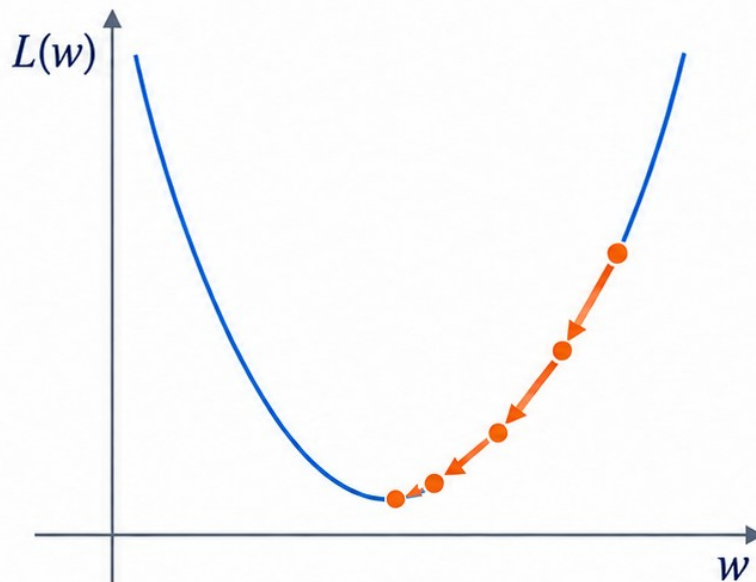
Выбор шага

Маленький шаг



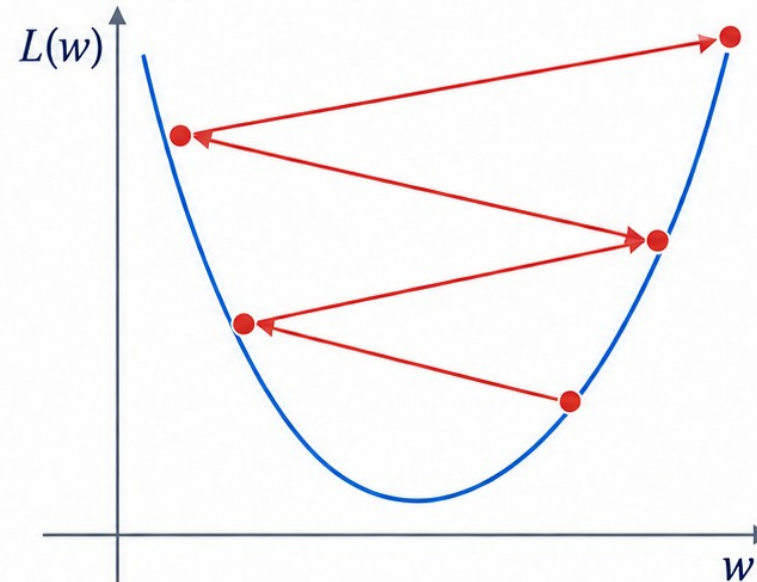
Медленная сходимость

Подходящий шаг



Быстрая сходимость

Слишком большой шаг



Расходимость

- ▶ **Маленький шаг:** параметры меняются слабо на каждой итерации, поэтому до минимума требуется много обновлений.
- ▶ **Подходящий шаг:** параметры заметно приближаются к минимуму. По мере приближения градиент уменьшается, а вместе с ним уменьшаются и обновления — алгоритм сходится.
- ▶ **Слишком большой шаг:** обновления перебрасывают параметры через минимум всё дальше от него. Колебания нарастают, значение функции потерь увеличивается — алгоритм расходится.

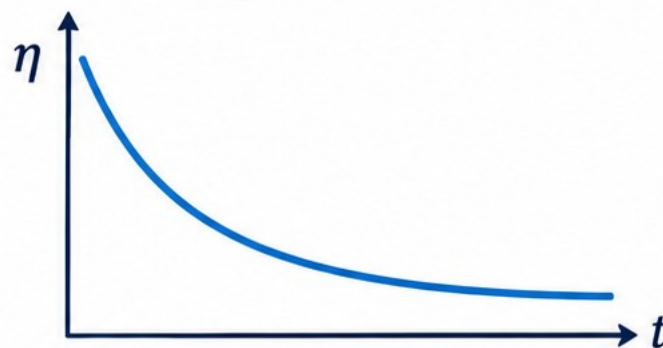
Выбор шага

Постоянный шаг



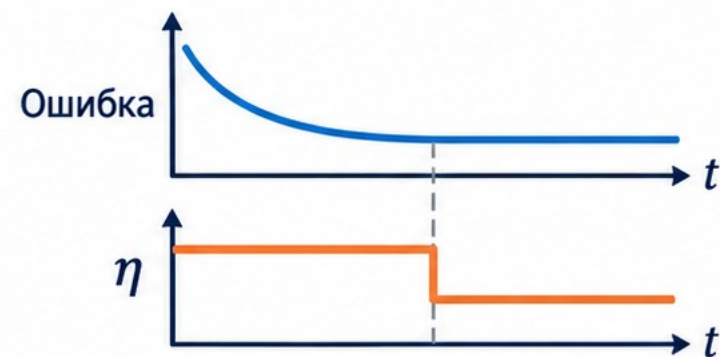
Подбираем η и фиксируем

По расписанию



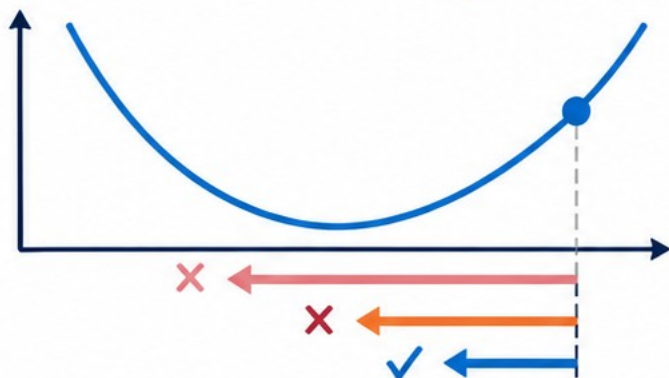
Уменьшаем η со временем

На плато



Снижаем η , если нет улучшений

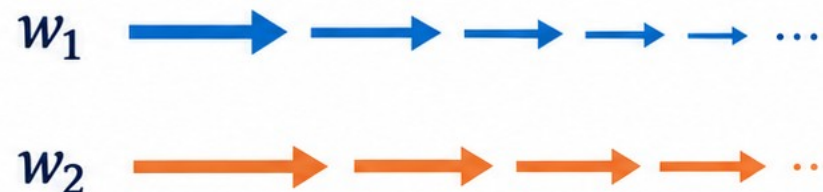
Поиск вдоль направления



Дробим шаг до достаточного убывания

Адаптивные методы

AdaGrad · RMSProp · Adam



Масштабируем шаг по истории градиентов

Для адаптивных методов базовый шаг тоже подбирают

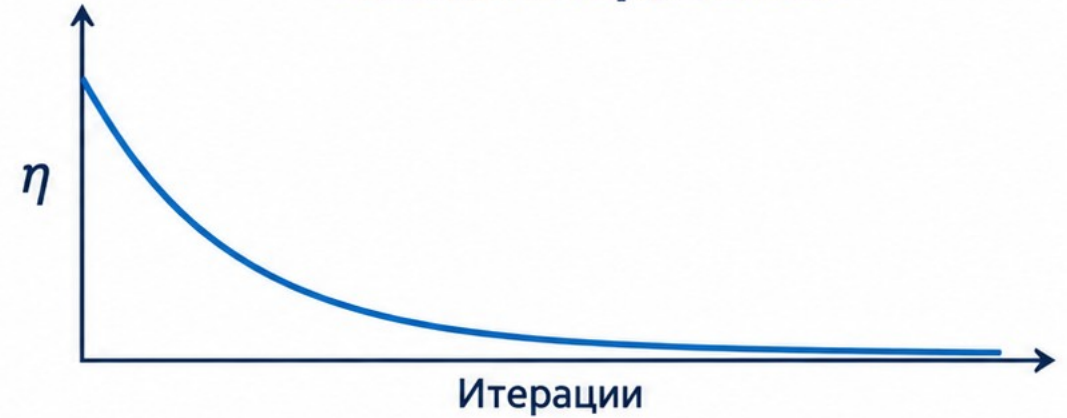
Выбор шага

Ступенчатое



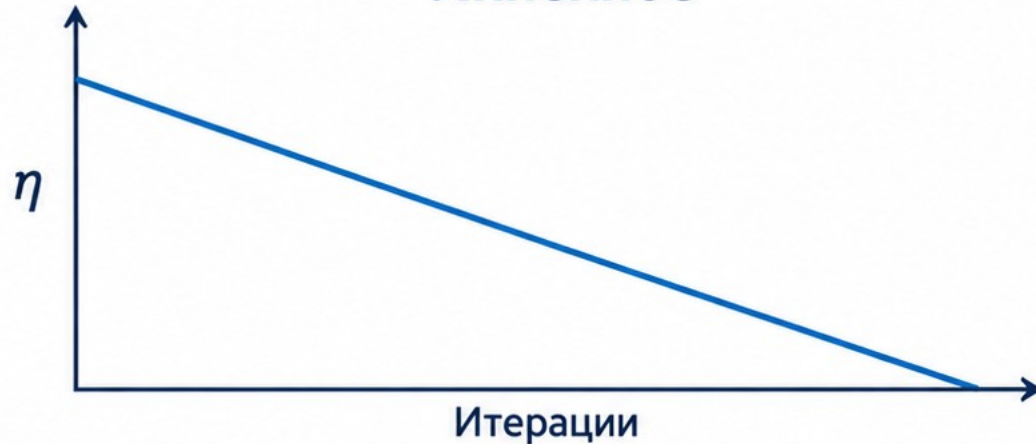
Снижаем шаг через заданное число итераций

Экспоненциальное



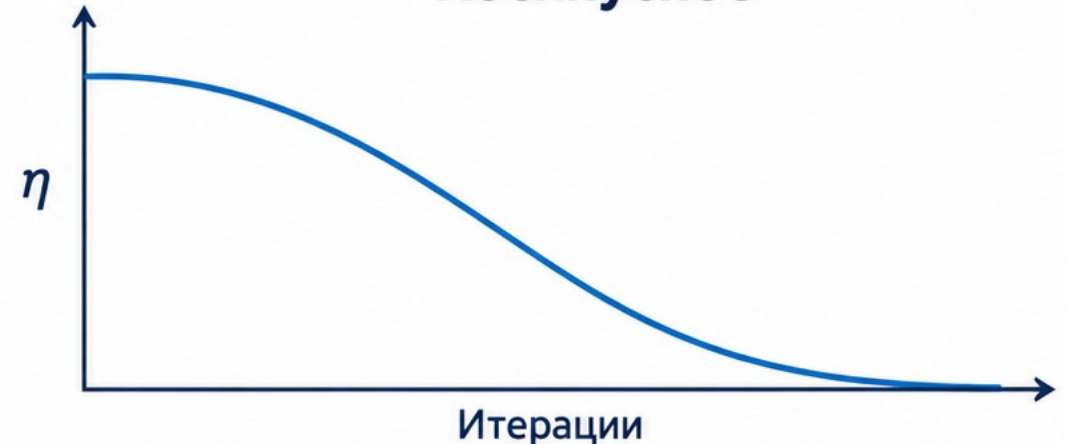
Умножаем шаг на постоянный коэффициент

Линейное



Уменьшаем шаг на одинаковую величину

Косинусное



Плавно снижаем шаг по полуволне косинуса

Выбор шага

В начале обучения большой шаг помогает быстрее приблизиться к минимуму. Позже меньший шаг позволяет точнее настраивать параметры, уменьшает колебания вокруг минимума и влияние шума стохастического градиента.

- ▶ **Ступенчатое:** даёт время на обучение с текущим шагом, затем переводит алгоритм к более точной настройке.
- ▶ **Экспоненциальное:** постепенно уменьшает относительный размер шага, обеспечивая непрерывный переход к небольшим обновлениям.
- ▶ **Линейное:** равномерно снижает шаг до конца заданного числа итераций; удобно при фиксированном бюджете обучения.
- ▶ **Косинусное:** сначала сохраняет большой шаг, затем снижает его быстрее и плавно выходит на малые значения для финальной настройки.

Слишком быстрое уменьшение шага может замедлить обучение раньше, чем алгоритм приблизится к минимуму.

Критерии остановки

Критерий	Условие	Смысл
Малая норма градиента	$\ \nabla L(w_t)\ < \varepsilon$	Функция почти не меняется при малых изменениях параметров
Малое изменение функции потерь	$ L(w_t) - L(w_{t-1}) < \varepsilon$	Дальнейшие итерации почти не улучшают результат
Малое изменение параметров	$\ w_t - w_{t-1}\ < \varepsilon$	Обновления стали незначительными
Исчерпан бюджет	Достигнут лимит итераций или времени	Ограничиваем вычислительные затраты
Нет улучшения на валидации	Ошибка на валидации не улучшается несколько проверок подряд	Останавливаем обучение до дальнейшего переобучения (early stopping)

Стохастический градиентный спуск

- ▶ Матрица наблюдений $X \in R^{n \times d}$ из n объектов, каждый из которых имеет d входных признаков:

$$X = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{pmatrix}$$

- ▶ Число строк – число наблюдений (объектов)
- ▶ Число столбцов – число входных признаков

Стохастический градиентный спуск

- ▶ Модель линейной регрессии:

$$\hat{y} = Xw$$

- ▶ Функция потерь MSE:

$$L(w) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

- ▶ Мы хотим минимизировать L :

$$L \rightarrow \min_w$$

- ▶ Градиентный спуск:

$$w^{(k+1)} = w^{(k)} - \eta \nabla L(w^{(k)})$$

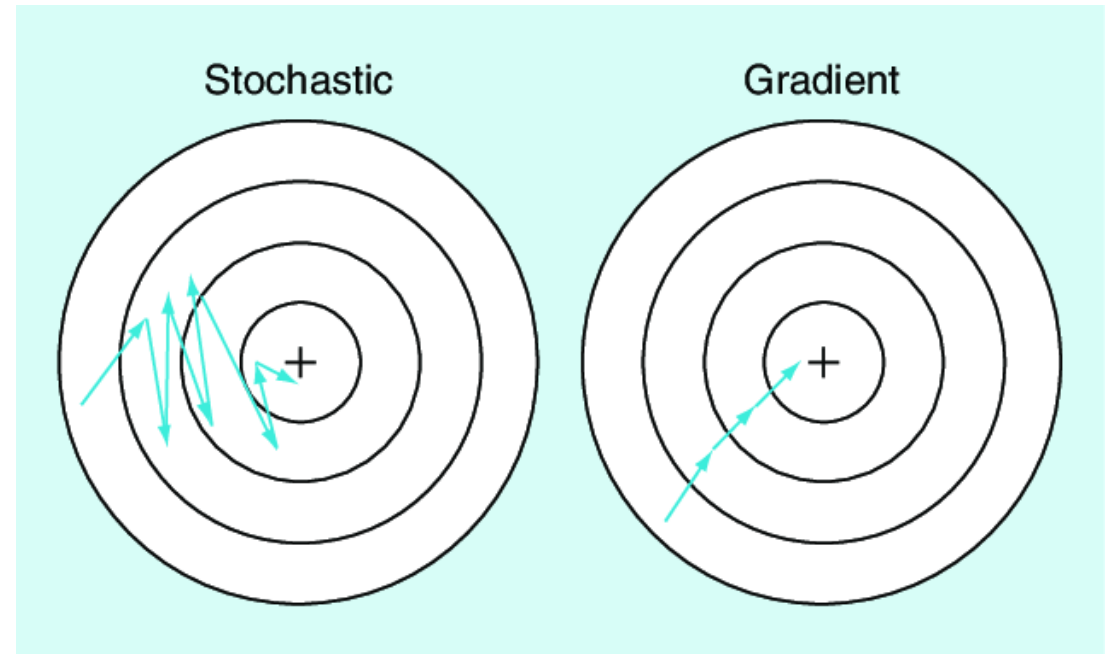
Стохастический градиентный спуск

- ▶ Полный градиентный спуск:

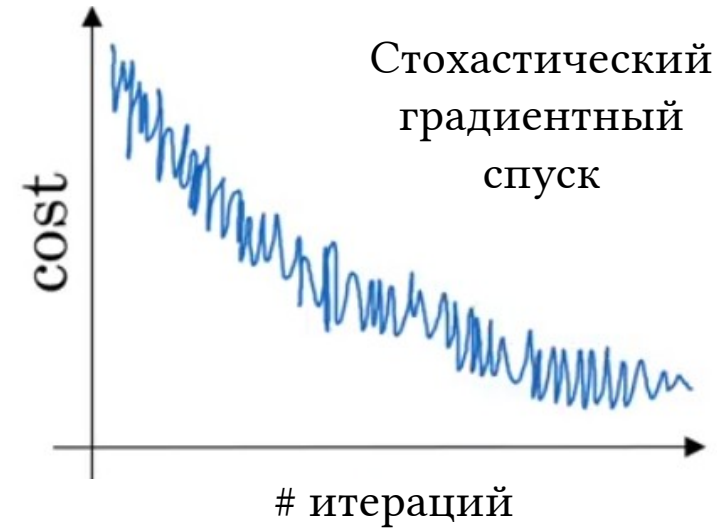
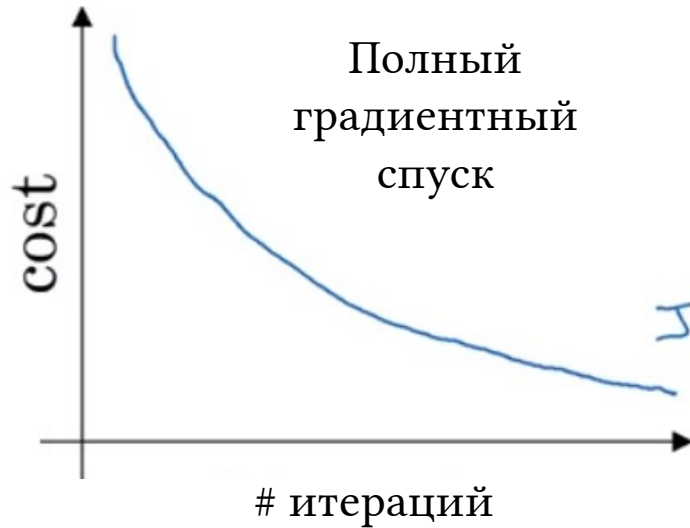
$$L(w) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$
$$w^{(k+1)} = w^{(k)} - \eta \nabla L(w^{(k)})$$

- ▶ **Стохастический** градиентный спуск:

$$L_i(w) = (\hat{y}_i - y_i)^2$$
$$w^{(k+1)} = w^{(k)} - \eta \nabla L_i(w^{(k)})$$



Стохастический градиентный спуск



- ▶ Стохастический ГС требует меньше вычислительных операций
- ▶ В полном ГС обучение стабильнее
- ▶ Полный ГС требует меньше итераций, но больше вычислительных операций

Стохастический градиентный спуск

- ▶ Полный градиентный спуск (gradient descent):

$$L(w) = \frac{1}{n} \sum_{i=1}^n L(\hat{y}_i, y_i)$$

- ▶ **Пакетный** градиентный спуск (batch gradient descent) ($1 < b < n$):

$$L(w) = \frac{1}{b} \sum_{i=1}^b L(\hat{y}_i, y_i)$$

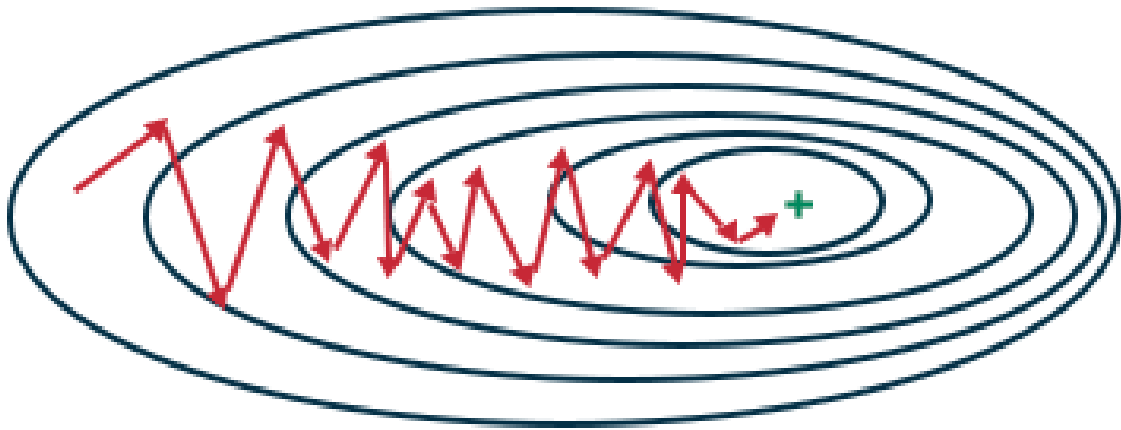
- ▶ **Стохастический** градиентный спуск (stochastic gradient descent):

$$L(w) = L(\hat{y}_i, y_i),$$

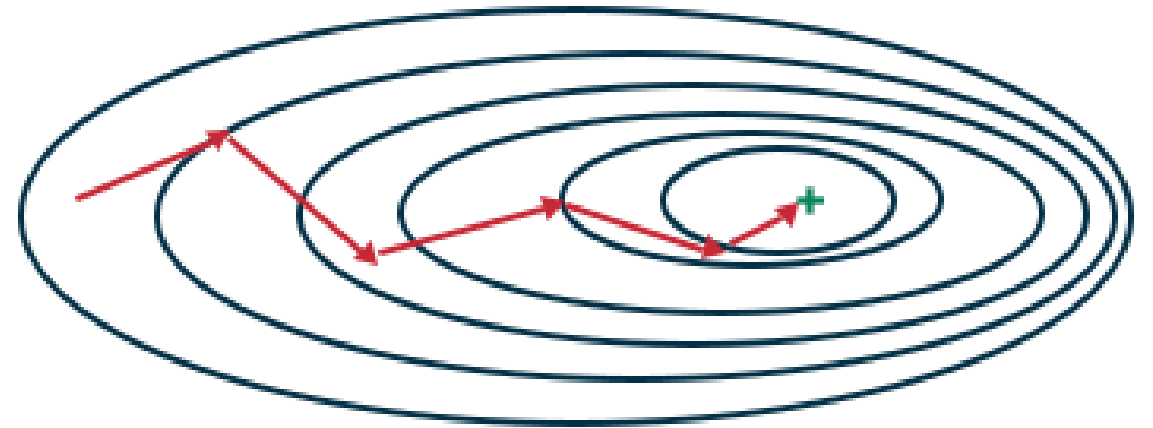
$$w^{(k+1)} = w^{(k)} - \eta \nabla L(w^{(k)})$$

Стохастический градиентный спуск

Stochastic Gradient Descent



Mini-Batch Gradient Descent



Эпохи и итерации

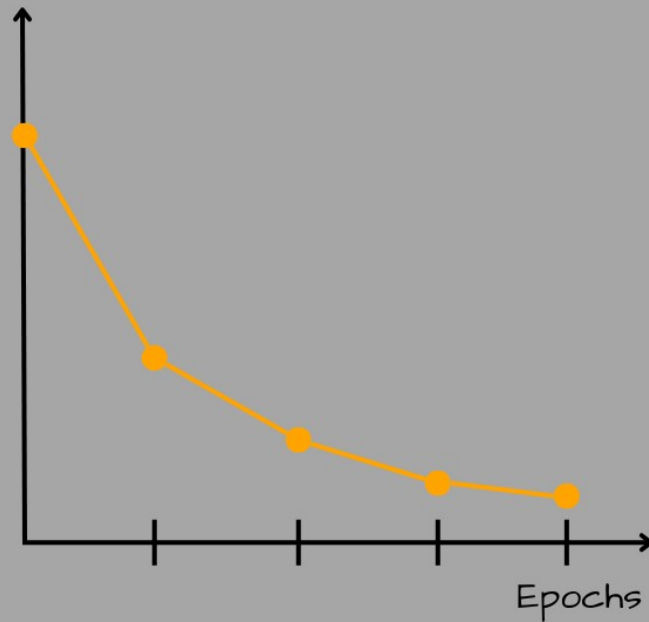
- ▶ **Итерация** – один шаг градиентного спуска (любого)
- ▶ **Эпоха** – это когда все объекты выборки прошли обучение

Пример: пусть обучающая выборка содержит 100 объектов. Тогда:

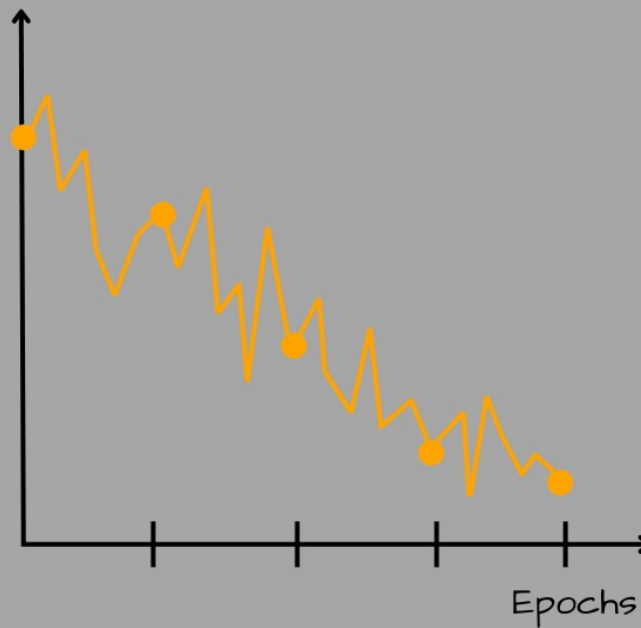
- ▶ Градиентный спуск (ГС): 1 эпоха = 1 итерация
- ▶ Пакетный ГС с `batch_size=10`: 1 эпоха = 10 итераций
- ▶ Стохастический ГС: 1 эпоха = 100 итераций

Стохастический градиентный спуск

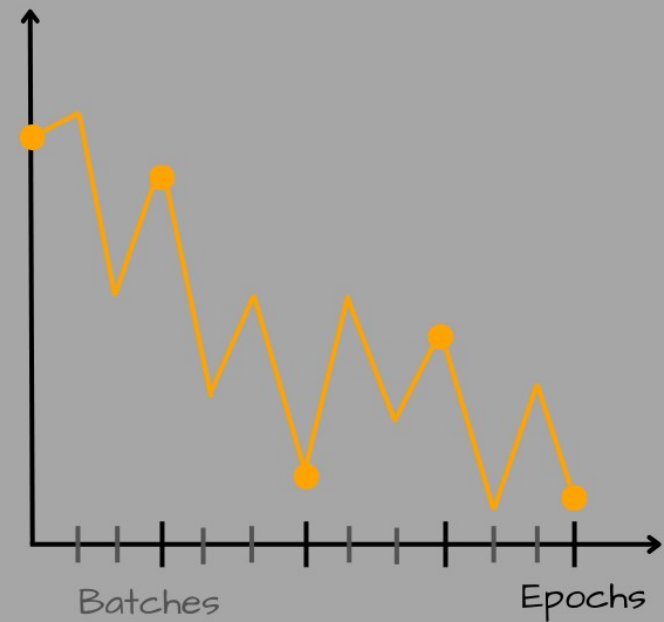
Convergence



Batch Gradient Descent



Stochastic Gradient Descent

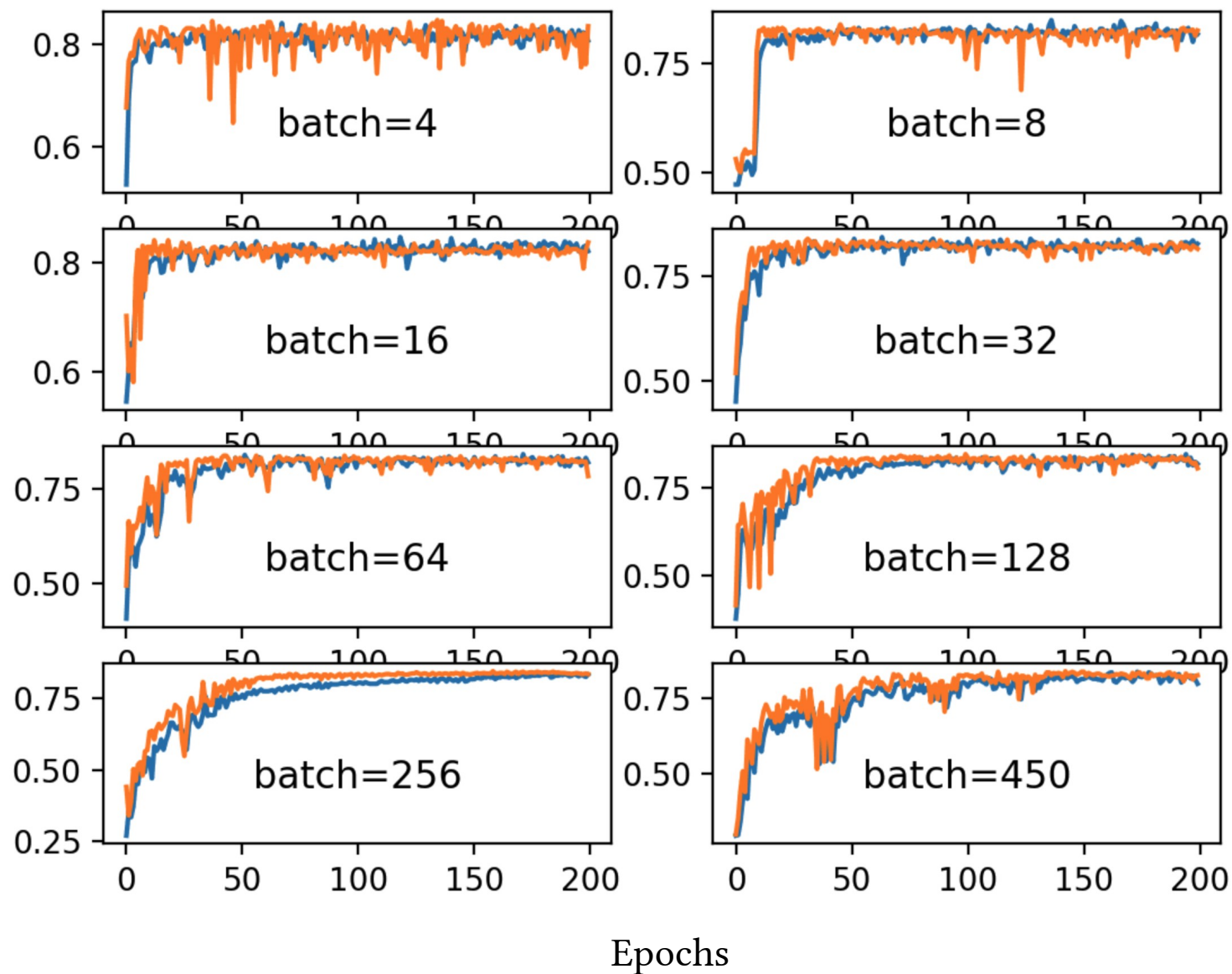


Mini Batch Gradient Descent

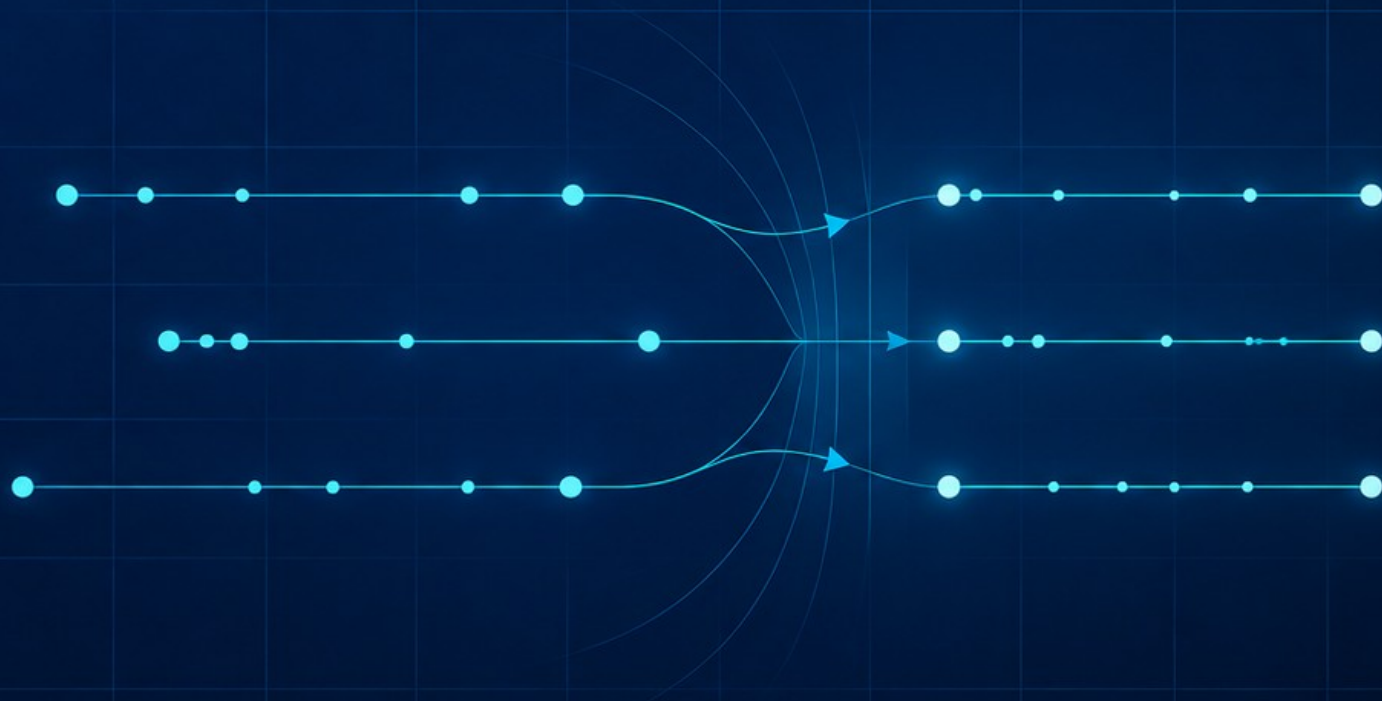
Datamap.ru

Пример:

Источник: <https://machinelearningmastery.com/how-to-control-the-speed-and-stability-of-training-neural-networks-with-gradient-descent-batch-size/>



Нормализация данных



Нормализация данных

- ▶ Модель линейной регрессии:

$$\hat{y}_i = w_0 + w_1 x_{i1} + w_2 x_{i2}$$

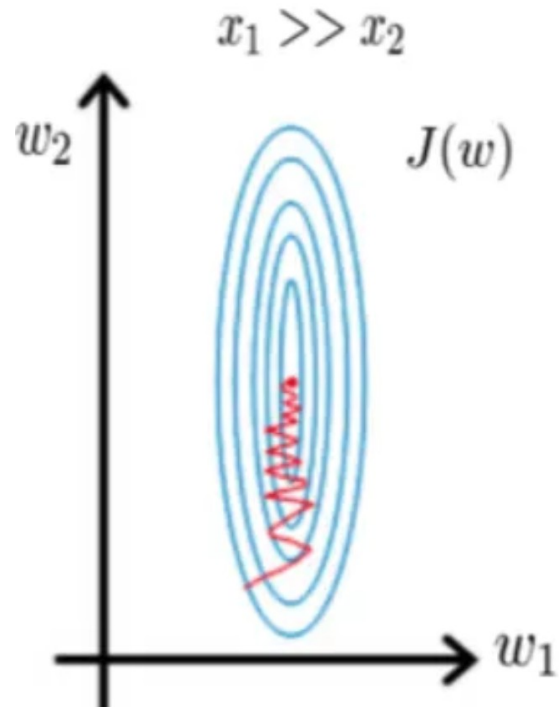
- ▶ Функция потерь MSE:

$$L(w) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \rightarrow \min_w$$

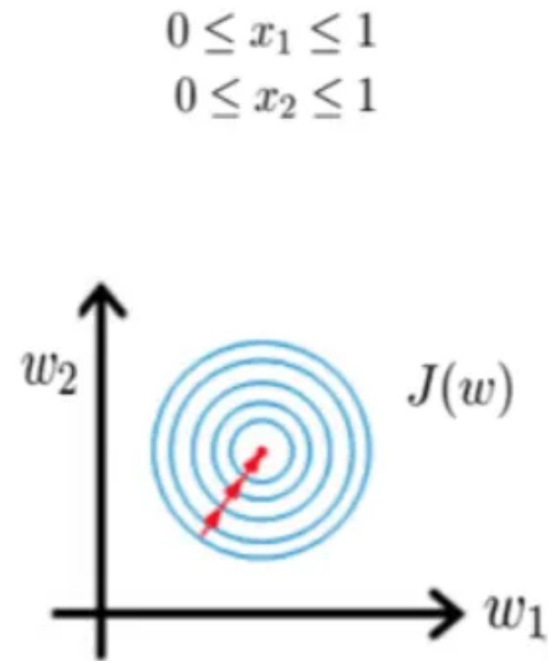
- ▶ Пусть признаки разные по масштабу:
 - например $|x_1| \approx 1000$, $|x_2| \approx 1$
- ▶ Тогда малые изменения dw_2 приводят к малым изменениям $L(w)$
- ▶ Малые изменения dw_1 приводят к **большим** изменениям $L(w)$

Нормализация данных

Gradient descent
without scaling



Gradient descent
after scaling variables



- ▶ Нормализация данных стабилизирует градиентный спуск
- ▶ Обучение происходит быстрее

Популярные способы нормализации

- ▶ Standard scaler:

$$x_i^{norm} = \frac{x_i - \mu}{\sigma}$$

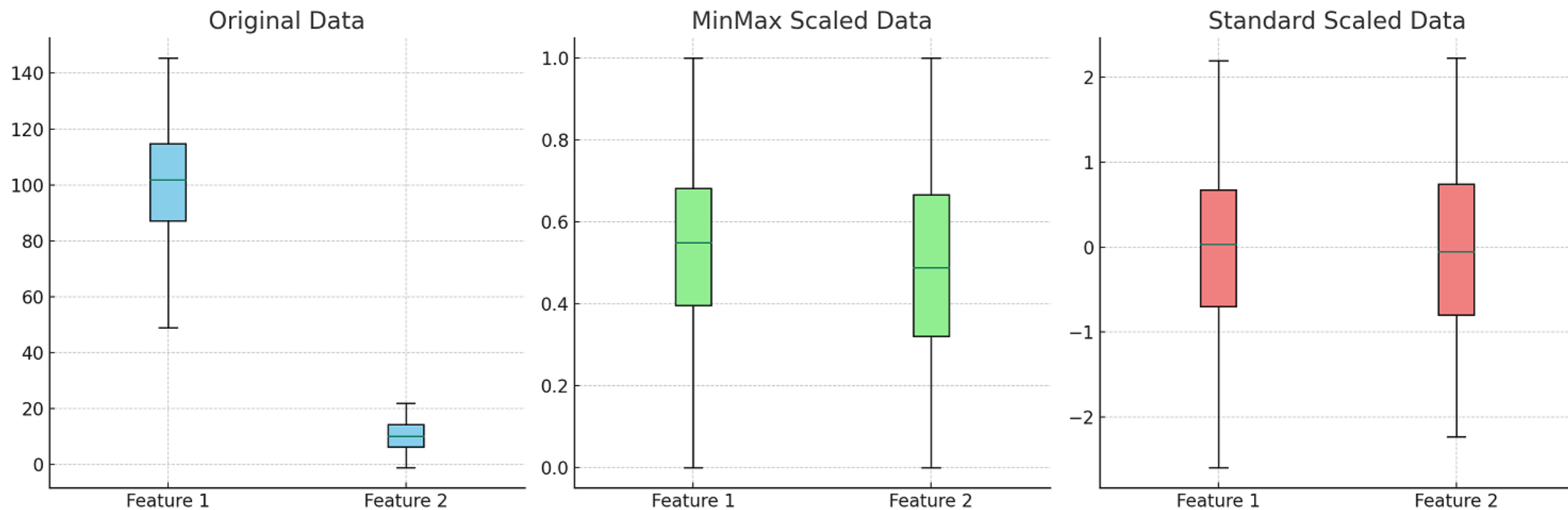
$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

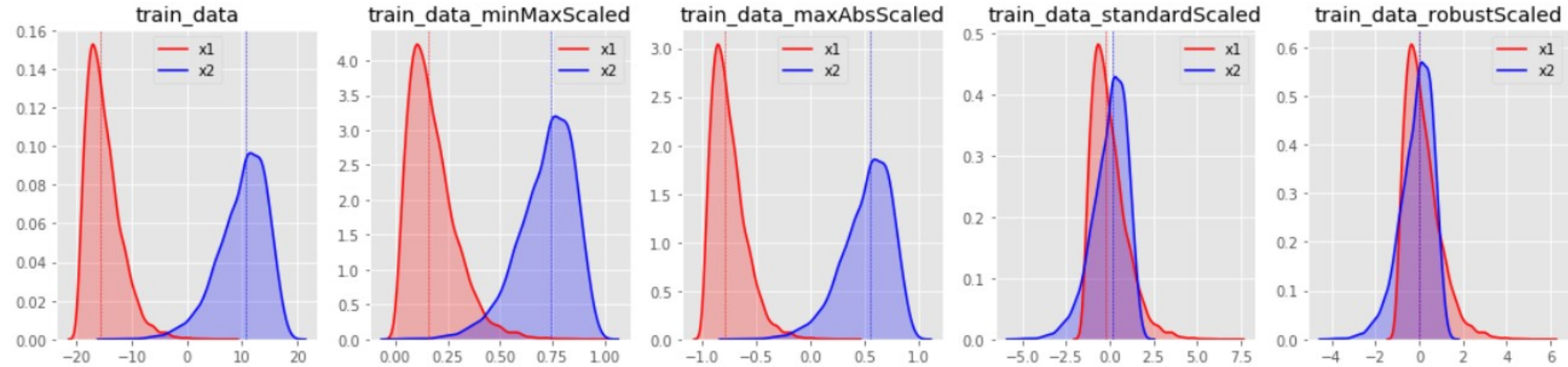
- ▶ Min-Max scaler:

$$x_i^{norm} = \frac{x_i - x_{min}}{x_{max} - x_{min}}$$

Популярные способы нормализации



Популярные способы нормализации

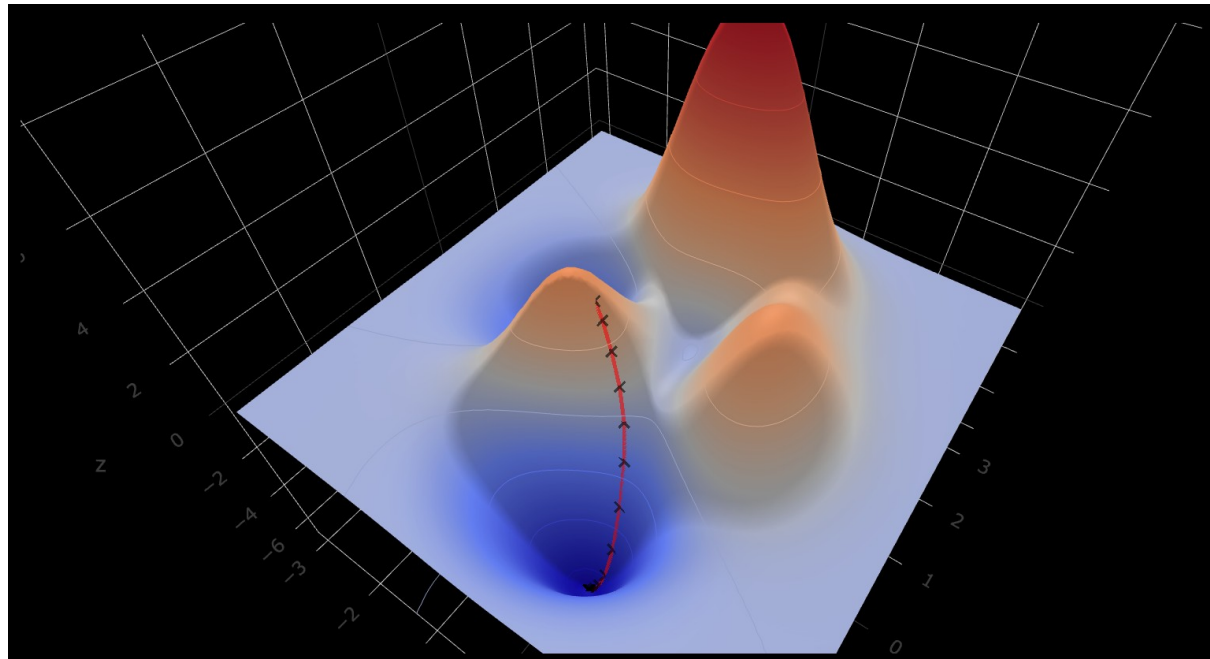


Заключение



Демонстрации

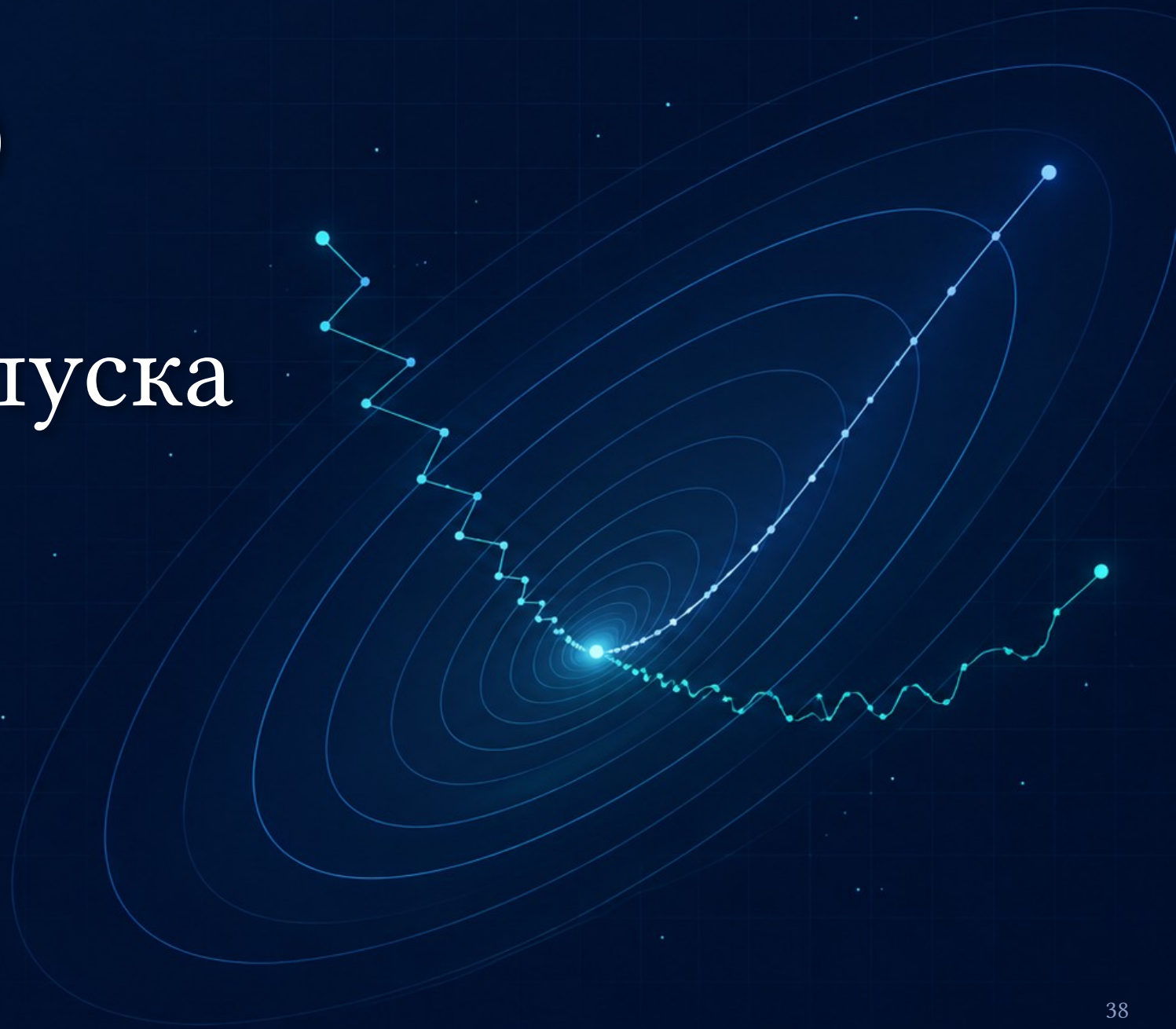
- ▶ Линейная регрессия с градиентным спуском <https://lukaszkujawa.github.io/gradient-descent.html>
- ▶ Градиентный спуск на сложной функции: <https://blog.skz.dev/gradient-descent>



Вопросы

- ▶ Что такое объект, целевая переменная, признак, модель, функция потерь, функционал ошибки и обучение?
- ▶ Что такое переобучение и недообучение? Как отличить переобучение от недообучения?
- ▶ Что такое кросс-валидация и для чего она используется? Чем применение кросс-валидации лучше, чем разбиение выборки на обучение и контроль?
- ▶ Чем гиперпараметры отличаются от параметров?
- ▶ Запишите формулы для линейной модели регрессии и для среднеквадратичной ошибки. Запишите среднеквадратичную ошибку в матричном виде.
- ▶ Что такое градиент? Какое его свойство используется при минимизации функций?
- ▶ Запишите алгоритм градиентного спуска. Приведите примеры критериев останова. Как длина шага влияет на процесс оптимизации?
- ▶ Для чего нужно нормировать данные при обучении линейных моделей? Какие способы нормировки вы знаете?
- ▶ Что такое регуляризация? Для чего ее используют в линейных моделях? Запишите L1- и L2-регуляризаторы. Почему L1-регуляризация отбирает признаки?

(Наобязательно) Модификации градиентного спуска



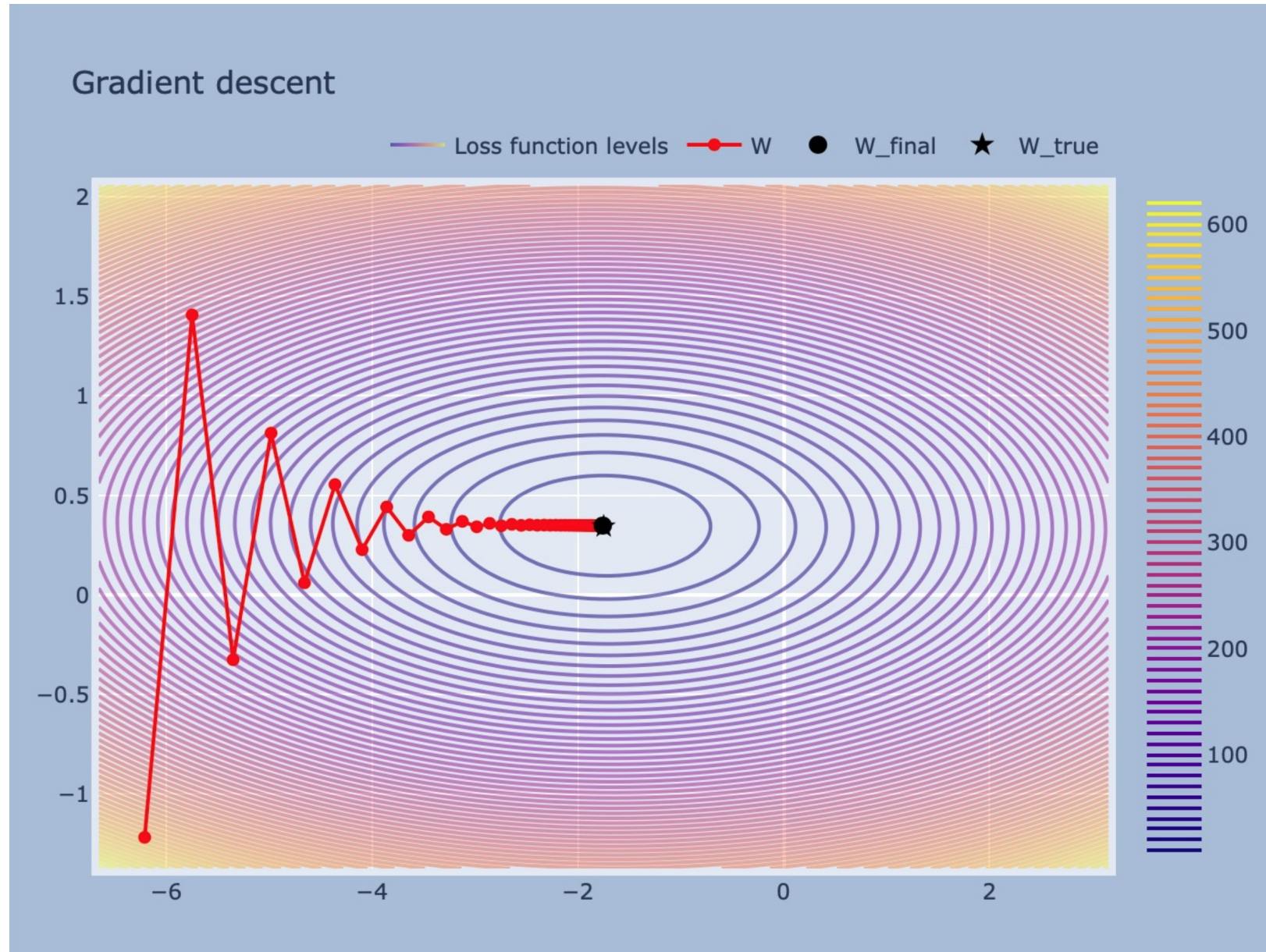
Проблемы градиентного спуска

- ▶ Рассмотрим функцию для оптимизации:

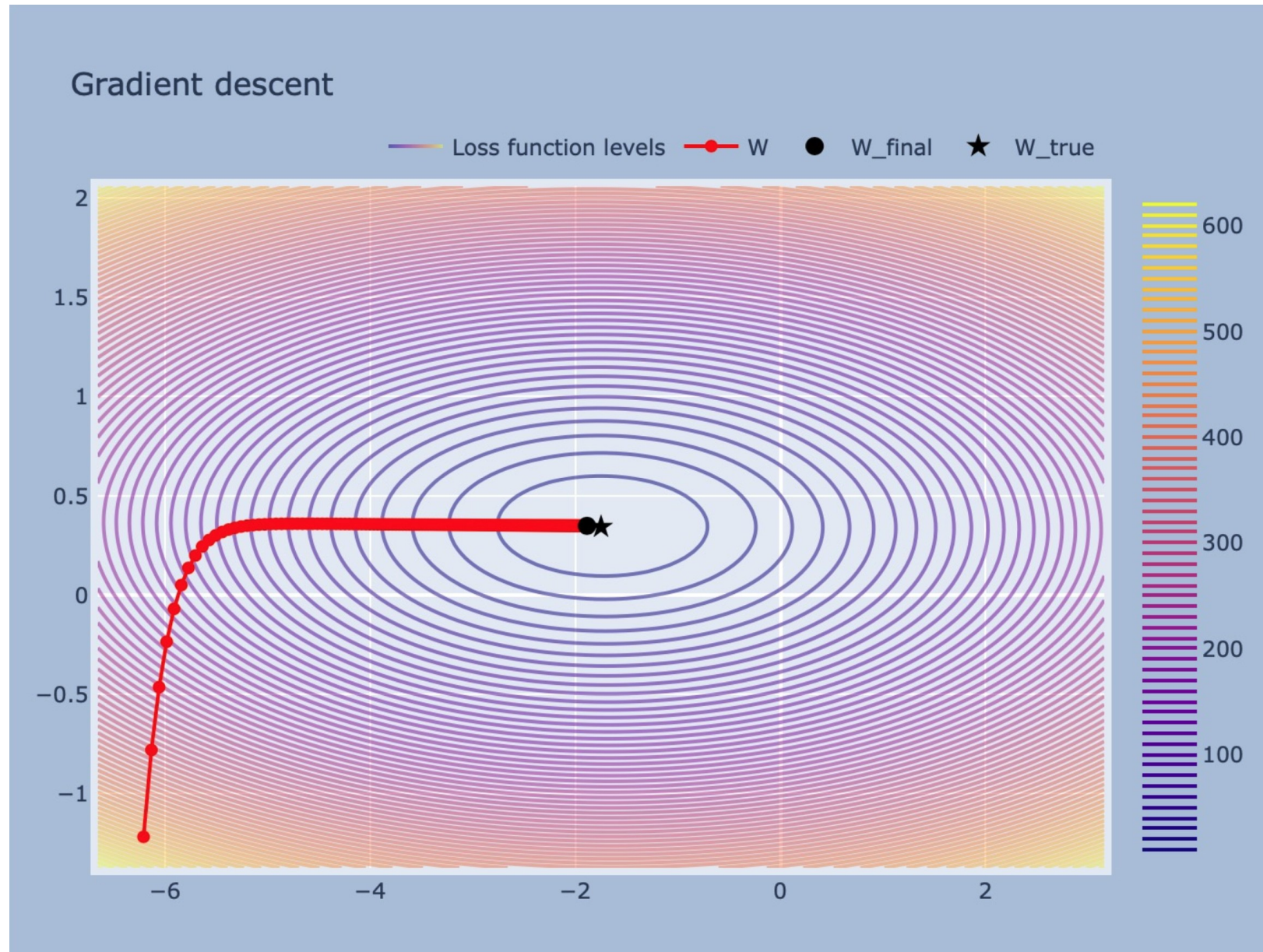
$$L(w) = 0.1w_1^2 + 2w_2^2$$

- ▶ Видим, что вклад w_2 сильно больше, чем для w_1
- ▶ Применим градиентный спуск

Проблемы градиентного спуска



Проблемы градиентного спуска



Проблемы градиентного спуска

- ▶ Если линии уровня функции вытянуты, то градиентный спуск требует аккуратного выбора длины шага
- ▶ Если шаг слишком большой, то градиентный спуск не сойдется
- ▶ Если шаг слишком маленький, то градиентный спуск будет сходиться слишком долго

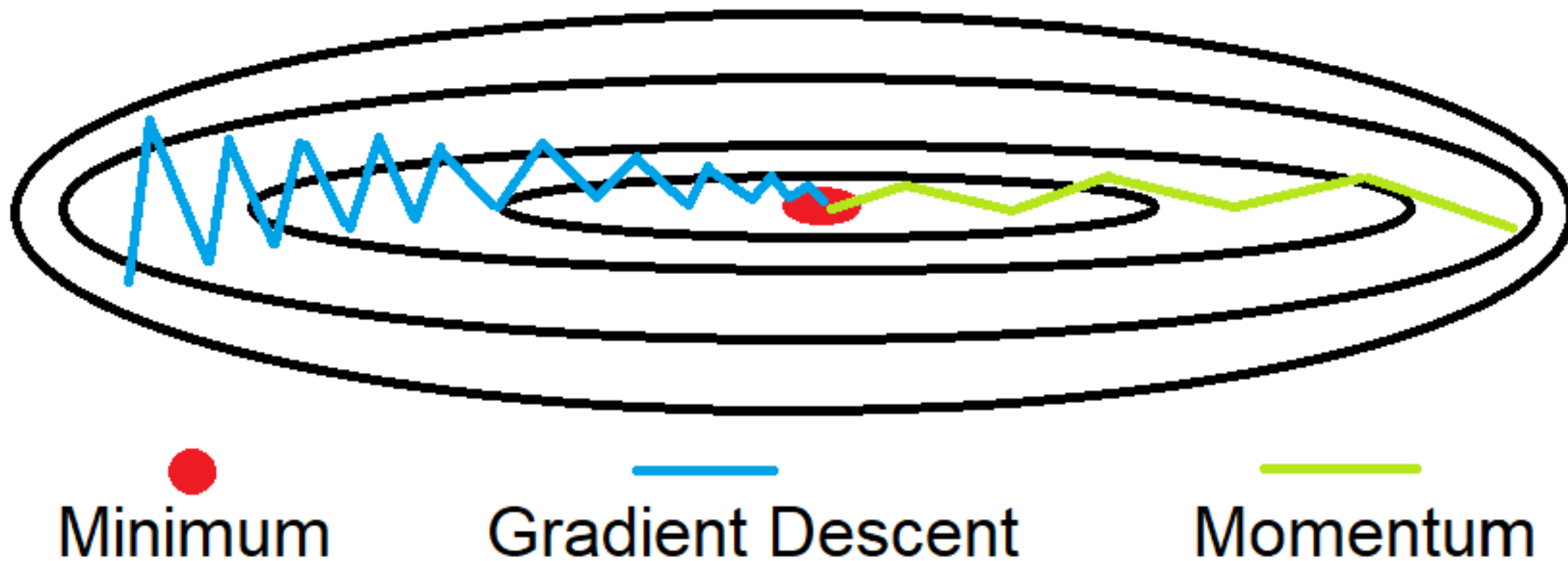
Momentum

- ▶
$$h_t = \beta h_{t-1} + \nabla L(w^{(t-1)})$$

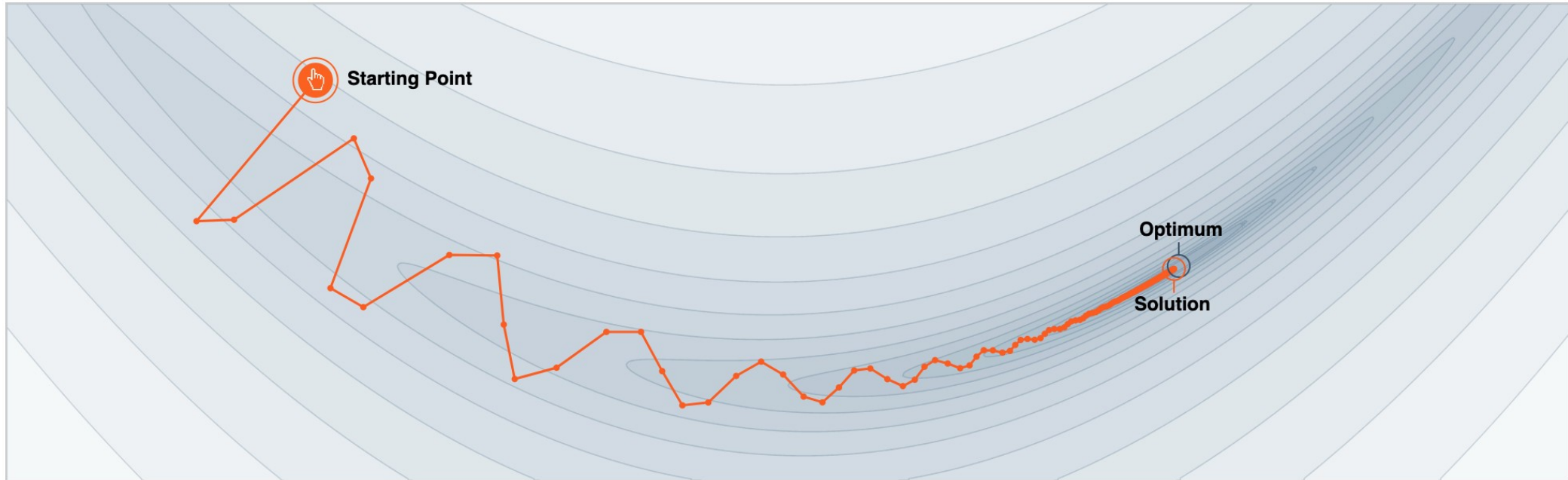
$$w^{(t)} = w^{(t-1)} - \eta h_t$$

- ▶ h_t — инерция, усредненное направление движения
- ▶ β — гиперпараметр затухания

Пример momentum



Пример momentum



Step-size $\alpha = 0.02$



Momentum $\beta = 0.83$



We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

<https://distill.pub/2017/momentum>

AdaGrad

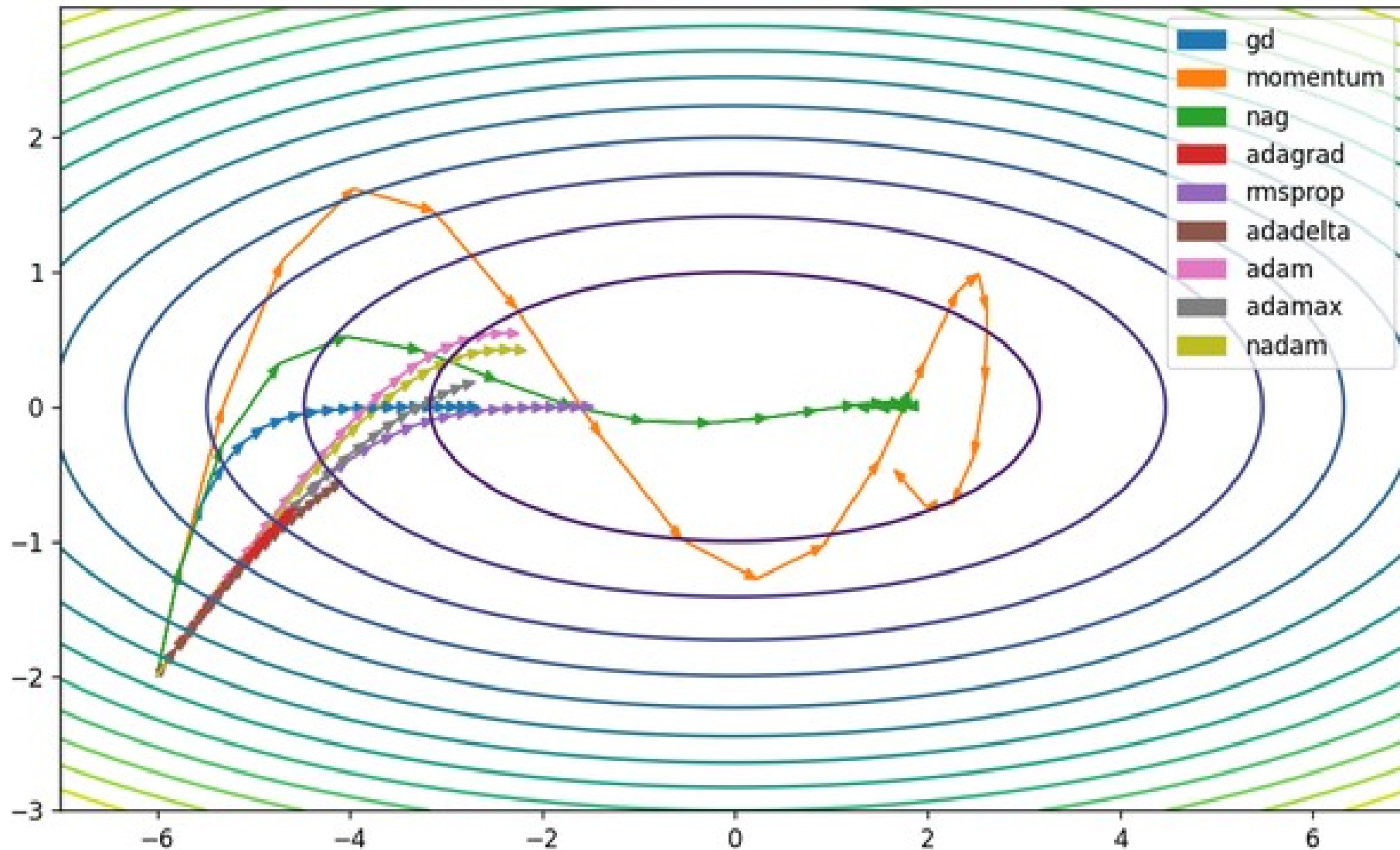
- ▶
$$g_{t-1} = \nabla L(w^{(t-1)})$$

$$s_t = s_{t-1} + (g_{t-1})^2$$

$$w^{(t)} = w^{(t-1)} - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_{t-1}$$

- ▶ $\odot$ - поэлементное умножение
- ▶ По каждому параметру (весу сети) своя скорость спуска. Это полезно, когда у признаков разный масштаб (1, 100, 1000)

Пример



RMSprop

- ▶
$$g_{t-1} = \nabla L(w^{(t-1)})$$

$$s_t = \gamma s_{t-1} + (1 - \gamma)(g_{t-1})^2$$

$$w^{(t)} = w^{(t-1)} - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot g_{t-1}$$

- ▶ $\gamma=0.9$
- ▶ Скорость спуска сильнее зависит от недавних шагов

Adam



$$g_{t-1} = \nabla L(w^{(t-1)})$$
$$h_t = \beta_1 h_{t-1} + (1 - \beta_1) g_{t-1}$$

$$\hat{h}_t = \frac{h_t}{1 - \beta_1^t}$$

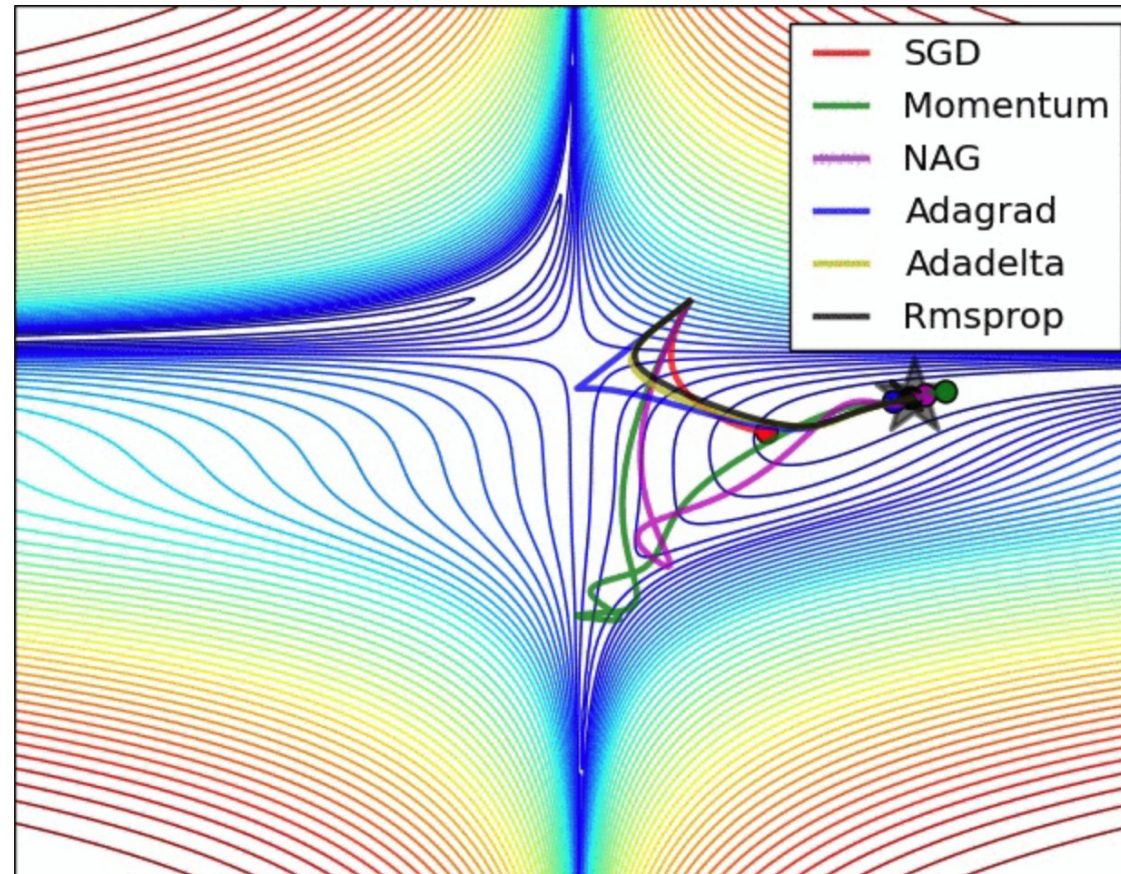
$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) (g_{t-1})^2$$

$$\hat{s}_t = \frac{s_t}{1 - \beta_2^t}$$

$$w^{(t)} = w^{(t-1)} - \frac{\eta}{\sqrt{\hat{s}_t} + \epsilon} \odot \hat{h}_t$$

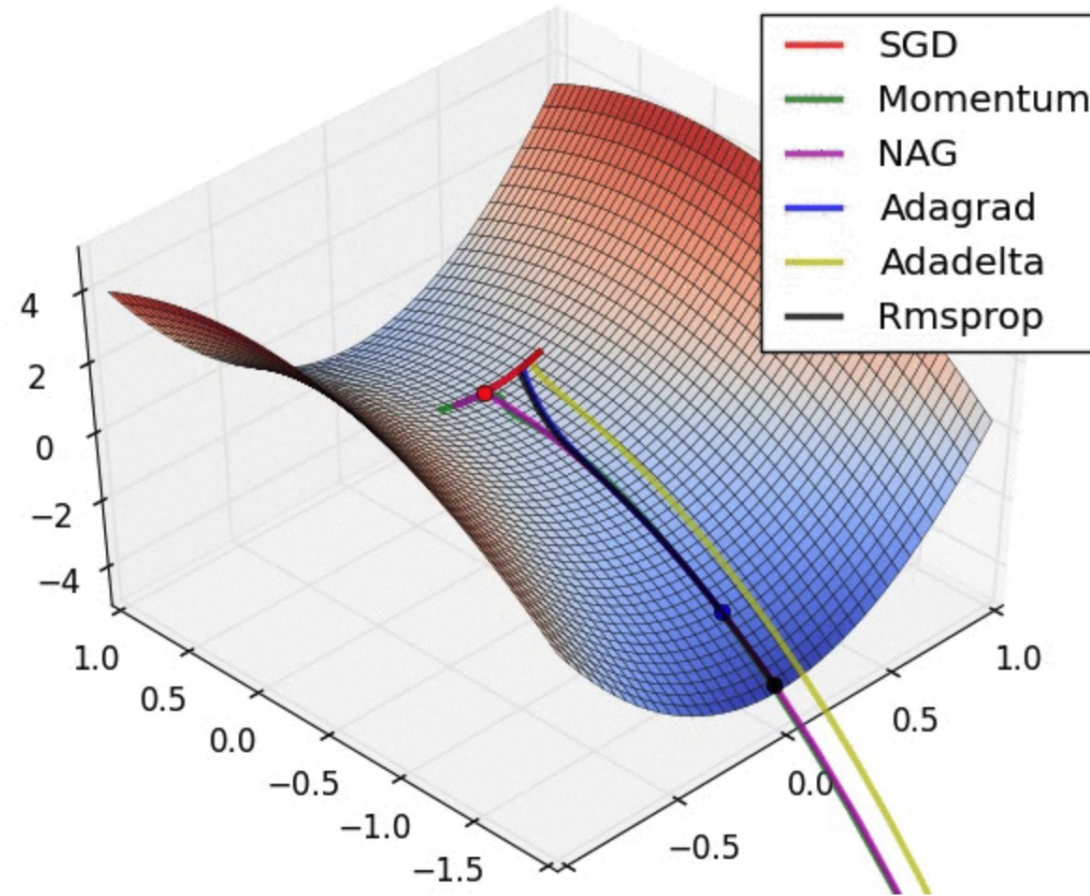
- ▶ $\beta_1 = 0.9$
- ▶ $\beta_2 = 0.999$

Пример



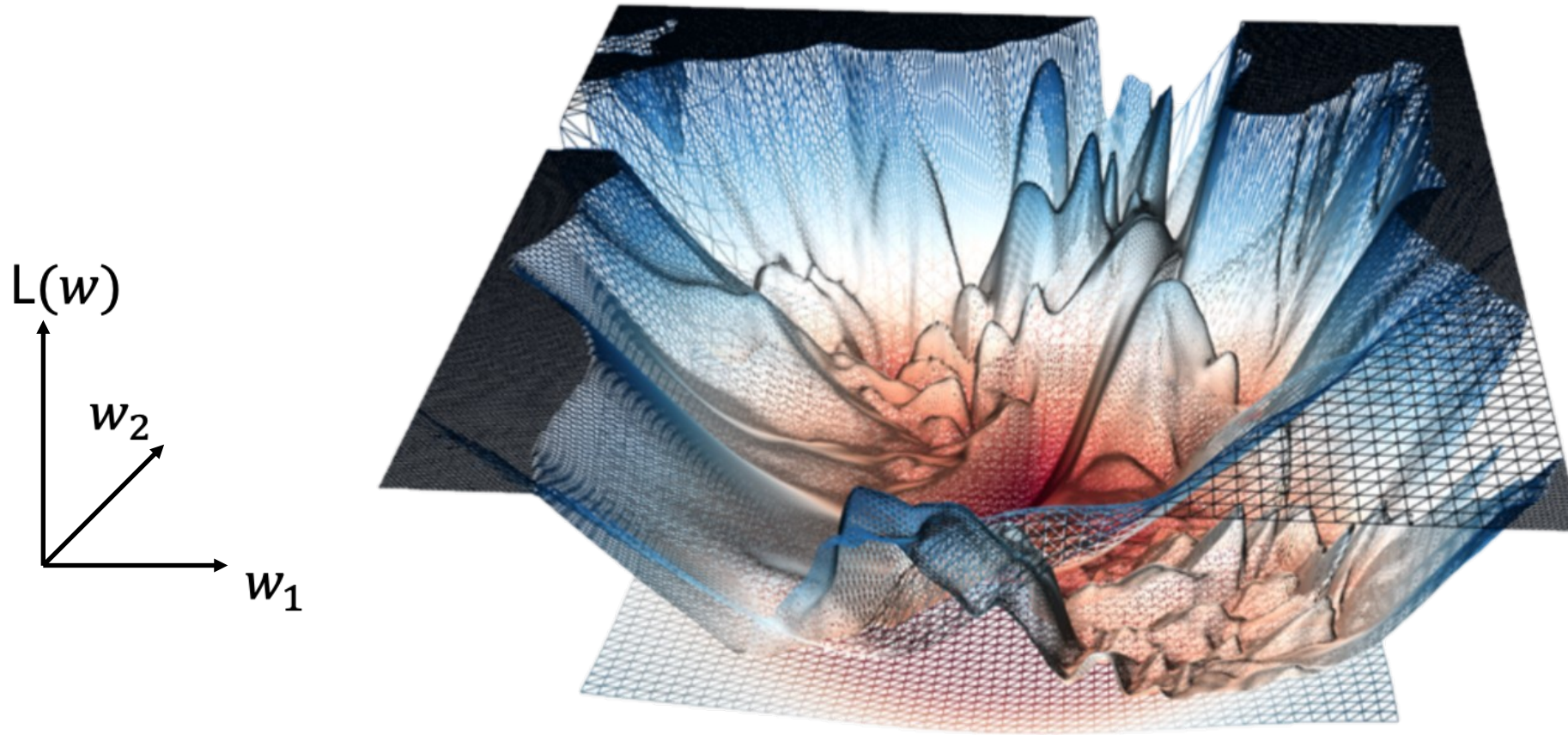
[https://api.wandb.ai/files/lavanyashukla/images/projects/36 993/d07b8a79.gif](https://api.wandb.ai/files/lavanyashukla/images/projects/36%20993/d07b8a79.gif)

Пример



<https://api.wandb.ai/files/lavanyashukla/images/projects/36993/2a6f771c.gif>

Примеры функций потерь нейронных сетей



- ▶ Demo: <https://www.telesens.co/loss-landscape-viz/viewer.html>
- ▶ Функции потерь как искусство: <https://losslandscape.com/>